Sweet Bites: código da aplicação (Situação-Problema 2)

🌐 **Aplicação no ar:** https://sweet-bites-s1we.onrender.com (o primeiro acesso pode levar cerca de 1 minuto, porque o servidor gratuito "dorme" quando ninguém usa).

📄 Versão em PDF deste manual: `pdf/README.pdf`.

🎬 **Vídeo narrado da solução em funcionamento (8min33s):** assistir no Google Drive

Loja virtual de cupcakes gourmet, com vitrine, carrinho, pedido, pagamento e acompanhamento da entrega, além de um painel do administrador. Foi construída a partir da especificação do PIT I (histórias de usuário, UML e protótipos).

Tecnologias

Camada	Tecnologia
Back-end	Java 17 + Spring Boot 4 (Spring MVC, Spring Security, Spring Data JPA)
Padrão de projeto	MVC: `model` (entidades), `controller`, `templates` (views); regras de negócio em `service`
Front-end	HTML5 + CSS3 + JavaScript puro, renderizados com Thymeleaf, layout mobile-first
Banco de dados	PostgreSQL 16, com migrations versionadas pelo Flyway
Testes	JUnit 5 + Mockito + Spring MockMvc (56 testes)
Hospedagem	Docker (Render), com o banco PostgreSQL no Neon

Estrutura

```
src/main/java/br/com/sweetbites
├── model/        entidades JPA (Usuario, Cupcake, Pedido, ItemPedido, Pagamento, Entrega...)
├── repository/   acesso ao banco (Spring Data JPA)
├── service/      regras de negócio (pedido, pagamento, frete, catálogo, usuário)
├── controller/   controllers MVC do cliente e do admin, e API JSON usada pelo JavaScript
├── config/       segurança, dados iniciais, configuração geral
└── job/          tarefa agendada (expiração do PIX)
src/main/resources
├── templates/    telas (Thymeleaf)
├── static/       CSS e JavaScript
└── db/migration/ scripts SQL do banco (Flyway)
src/test/java     testes unitários e de integração
```

Como executar localmente

Pré-requisitos: Java 17 e Docker.

```
docker compose up -d                        # sobe o PostgreSQL na porta 5433
cp .env.example .env                        # ajuste ADMIN_EMAIL e ADMIN_SENHA
set -a; . ./.env; set +a
./mvnw spring-boot:run                      # http://localhost:8080
```

No primeiro boot, o sistema cria as tabelas, o cardápio de exemplo (10 cupcakes em 5 categorias) e o usuário administrador, com o e-mail e a senha definidos em `ADMIN_EMAIL` / `ADMIN_SENHA`.

Para rodar os testes, que usam banco H2 em memória e não precisam de Docker:

```
./mvnw test
```

Manual de uso

Cliente

1. **Criar conta** (US-01): toque em *Cadastre-se* e informe nome, e-mail, telefone e senha (mínimo de 8 caracteres, com letras e números).
2. **Entrar** (US-02): use e-mail e senha. Depois de 5 tentativas erradas a conta é bloqueada; para desbloquear, use *Esqueci minha senha*. Para trocar a senha estando logado, use **Perfil → Alterar senha**.
3. **Recuperar senha** (US-18): informe o e-mail e use o link recebido, válido por 1 hora. Ao redefinir, todas as sessões abertas são encerradas.
4. **Vitrine** (US-04/05): filtre por categoria ou busque por nome ou ingrediente. A busca acontece enquanto você digita. Cupcakes sem estoque aparecem como *Indisponível*.
5. **Detalhes** (US-06): toque no cupcake para ver descrição, ingredientes e preço, escolher a quantidade e adicionar ao carrinho.
6. **Carrinho** (US-07): altere quantidades (máximo de 20 por item) ou remova itens. O carrinho continua salvo mesmo fechando o navegador.
7. **Finalizar pedido** (US-08/09): informe o CEP (o endereço e o frete são calculados automaticamente), o número e a forma de pagamento: cartão (até 3x sem juros), PIX ou dinheiro na entrega. O pedido mínimo é de R$ 15,00.
8. **Acompanhar** (US-10): a tela do pedido mostra o status em tempo real (atualiza sozinha) e cada mudança gera uma notificação no sino do topo.

Administrador

Entre com o e-mail de administrador. Você cai direto no **Painel**.

- **Painel**: pedidos por status e alerta de estoque baixo (menos de 5 unidades).
- **Pedidos** (US-16): filtre por status e período. Abra o pedido para avançar o status (*Em preparação → Saiu para entrega → Entregue*) ou cancelar. O cliente é notificado a cada mudança. Pedidos entregues não podem ser cancelados.
- **Cupcakes** (US-14): cadastre, edite ou remova cupcakes, com foto em JPG/PNG de até 5MB. Cupcakes em pedidos em andamento não podem ser removidos.
- **Estoque** (US-15): ajuste a quantidade disponível. O estoque baixa sozinho a cada pedido e volta quando um pedido é cancelado.
- **Categorias**: crie ou remova categorias (só é possível remover categorias sem cupcakes).

Integrações simuladas (modo demonstração)

Para o projeto acadêmico não ter custo, as integrações pagas são **simuladas**. Com `MODO_DEMONSTRACAO=true`, a interface avisa sempre que algo é simulação.

Integração	Como funciona aqui
Consulta de CEP	**Real**: API pública ViaCEP
Cartão de crédito	Gateway simulado: o número precisa ser válido (algoritmo de Luhn) e a validade futura. O cartão de teste `4111 1111 1111 1111` é aprovado; cartões terminados em `0002` (ex.: `4000 0000 0000 0002`) são recusados, o que demonstra o fluxo alternativo Beta do UC-08. Só os 4 últimos dígitos são guardados.
PIX	QR Code e código "copia e cola" fictícios, válidos por 30 minutos. O botão *Simular pagamento do PIX* confirma o pagamento. Sem pagamento, o pedido é cancelado automaticamente e o estoque volta.
E-mail (boas-vindas, recuperação de senha, pedido)	Registrado no log do servidor. Na recuperação de senha, o link aparece na tela.
Notificação push	Notificações dentro do app (ícone de sino), com a tela do pedido atualizando o status a cada 10 segundos.

Regras de frete (configuráveis em `application.properties`)

A loja fica em São Paulo/SP. A distância é aproximada por zona:

Destino	Frete	Tempo estimado
Cidade de São Paulo	R$ 7,00	45 min
Outras cidades de SP	R$ 15,00	120 min
Outros estados	Sem entrega	—

Variáveis de ambiente (produção)

Variável	Descrição
`DATABASE_URL`	URL JDBC do PostgreSQL, ex.: `jdbc:postgresql://host/banco?sslmode=require`
`DATABASE_USER` / `DATABASE_PASSWORD`	Credenciais do banco
`ADMIN_EMAIL` / `ADMIN_SENHA`	Administrador criado no primeiro boot
`MODO_DEMONSTRACAO`	`true` exibe os atalhos de simulação (PIX e link de senha)
`PORT`	Porta HTTP (o Render define sozinho)

Segurança

- Senhas guardadas com BCrypt. Bloqueio após 5 tentativas erradas.
- Proteção CSRF em todos os formulários, e Content-Security-Policy sem scripts inline.
- Áreas `/admin/**` liberadas só para o papel ADMIN.

- Um cliente não acessa pedido de outro: a resposta é 404, igual a um pedido inexistente.
- Preços e estoque são sempre recalculados no servidor; o carrinho do navegador não é confiável.
- Upload de imagem validado pelo conteúdo do arquivo, não só pela extensão.
- Token de recuperação de senha guardado como hash SHA-256, de uso único e com validade de 1 hora.